

Practice 4

Name: Jitesh Choudhary

PRN: 1272240760

```
# 2. Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
# 3. Load Titanic Dataset
data = pd.read_csv("./train.csv")
```

```
# 4. Select useful features
data = data[["Pclass", "Sex", "Age", "Fare", "Embarked", "Survived"]]
```

```
# 5. Display Dataset Information
print(data.info())
print(data.head())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Pclass      891 non-null    int64
1   Sex         891 non-null    str
2   Age         714 non-null    float64
3   Fare        891 non-null    float64
4   Embarked    889 non-null    str
5   Survived    891 non-null    int64
dtypes: float64(2), int64(2), str(2)
memory usage: 41.9 KB
None
```

	Pclass	Sex	Age	Fare	Embarked	Survived
0	3	male	22.0	7.2500	S	0
1	1	female	38.0	71.2833	C	1
2	3	female	26.0	7.9250	S	1
3	1	female	35.0	53.1000	S	1
4	3	male	35.0	8.0500	S	0

```
# 6. Handle missing Age, Fare and Embarked values
data["Age"] = data["Age"].fillna(data["Age"].mean())
data["Fare"] = data["Fare"].fillna(data["Fare"].mean())
data["Embarked"] = data["Embarked"].fillna(data["Embarked"].mode()[0])
```

```
# 7. Convert categorical values using dummy variables
data = pd.get_dummies(data, columns=["Sex", "Embarked"], drop_first=True)
```

```
# 8. Split data into 80% training and 20% testing
X = data.drop("Survived", axis=1)
y = data["Survived"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 9. Create Logistic Regression Model
model = LogisticRegression(max_iter=1000)
```

```
# 10. Train the model
model.fit(X_train, y_train)
```

▼ **LogisticRegression** ⓘ ?

► Parameters

► Fitted attributes

```
# 11. Predict survival for test data
y_pred = model.predict(X_test)
```

```
# 12. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.7988826815642458

```
# 13. Calculate survival probabilities
survival_probabilities = model.predict_proba(X_test)[: , 1]
```

```
# 14. Display actual and predicted values
result = pd.DataFrame({
    "Actual Survival": y_test.values,
    "Predicted Survival": y_pred,
    "Survival Probability": survival_probabilities
})

print(result.head(20))
```

	Actual Survival	Predicted Survival	Survival Probability
0	1	0	0.140224
1	0	0	0.204565
2	0	0	0.110088
3	1	1	0.850571
4	1	1	0.738821
5	1	1	0.905275
6	1	1	0.635804
7	0	0	0.120743
8	1	1	0.711706
9	1	1	0.918140
10	0	0	0.380653
11	0	0	0.063161
12	0	1	0.531958
13	0	0	0.138998
14	0	0	0.184875
15	1	1	0.923963
16	1	0	0.351280